



MARS

A MIPS32 Simulator

Introduction

- ***MARS*** (**M**IPS **A**ssembler and **R**untime **S**imulator) is an IDE for MIPS Assembly Language Programming
- MARS is a lightweight interactive development environment (IDE) for programming in MIPS assembly language, intended for educational-level use

Features

- GUI with point-and-click control and integrated editor
- Easily editable register and memory values, similar to a spreadsheet
- Display values in hexadecimal or decimal
- Variable-speed single-step execution
- Single-step backwards

Installation

- Installation Link -
<http://courses.missouristate.edu/KenVollmar/MARS/download.htm>
- Start up the MARS simulator by running/opening (double-clicking) the file you downloaded, Mars_4_2.jar.
- It is a Java executable, so you should have java (i.e. has the Java JRE or JDK) installed on your computer

User Interface

The screenshot displays the Mars MIPS assembler simulator interface. The main window is titled "Edit / Execute" and shows the assembly code for a program named "sums.s". The code is as follows:

```
1  # This program computes and displays the sum of integers from 1 up to n,
2  # where n is entered by the user.
3  #
4
5  .globl    main
6
7  .data
8
9  # program output text constants
10 prompt:
11  .asciiz  "Please enter a positive integer: "
12 result1:
13  .asciiz  "The sum of the first "
14 result2:
15  .asciiz  " integers is "
16 newline:
17  .asciiz  "\n"
18
19 .text
20
21 # main program
22 #
23 # program variables
24 # n: $s0
25 # sum: $s1
```

The status bar at the bottom of the main window indicates "Line: 14 Column: 9" and "Show Line Numbers" is checked.

On the right side, the "Registers" window is open, showing a table of registers and their values:

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000000
\$v1	3	0x00000000
\$a0	4	0x00000000
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000000
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000000
\$s1	17	0x00000000
\$s2	18	0x00000000
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x00000000
\$t9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7ffefffc
\$fp	30	0x00000000
\$ra	31	0x00000000
pc		0x00400000
hi		0x00000000
lo		0x00000000

At the bottom, the "Mars Messages / Run I/O" window is open, showing a "Clear" button.

Menu Bar

- The menu bar has menu items for performing various actions.
- The most useful menus for assembly language programming are the "File", "Edit", "Run", and "Help" menus.
- The most commonly used menu items are available on the toolbar.

Tool Bar

- The tool bar contains buttons for the most commonly used menu items.
- They are grouped into four groups corresponding to the "File", "Edit", "Run", and "Help" menus.

File Menu



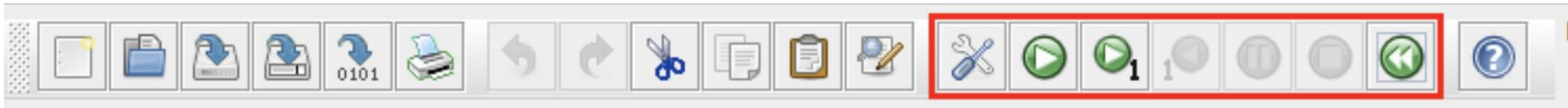
Menu Item	Icon	Accel	Description
New		Ctrl-N	Create a new empty file
Open ...		Ctrl-O	Open a file
Close		Ctrl-W	Close the current file
Close All			Close all files
Save		Ctrl-W	Save the current file
Save as ...		Ctrl-S	Save the current file with a new name
Save All			Save all assembly language files
Dump Memory		Ctrl-D	Create a file dump of the text or data section for the current loaded program
Print ...			Print the current file
Exit			Exit the Mars program

Edit Menu



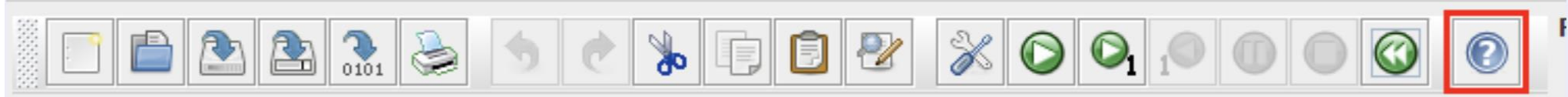
Menu Item	Icon	Accel	Description
Undo		Ctrl-Z	Undo a previous edit
Redo		Ctrl-Y	Redo a previously undone edit
Cut		Ctrl-X	Cut the currently selected text
Copy		Ctrl-C	Copy the currently selected text
Paste		Ctrl-V	Replace the current text selection with the previous cut or copy text
Find/Replace		Ctrl-F	Find or replace text
Select All		Ctrl-A	Select all text in the current file

Run Menu



Menu Item	Icon	Accel	Description
Assemble		F3	Assemble and load the current file
Go		F5	Run the current loaded program
Step		F7	Run a single instruction of the current loaded program
Backstep		F8	Undo the last executed instruction
Pause		F9	Pause execution of the current loaded program
Stop		F11	Stop execution of the current loaded program
Reset		F12	Reload the current assembled file
Clear All Breakpoints		Ctrl-K	Clear all breakpoints
Toggle All Breakpoints		Ctrl-T	Toggle all breakpoints

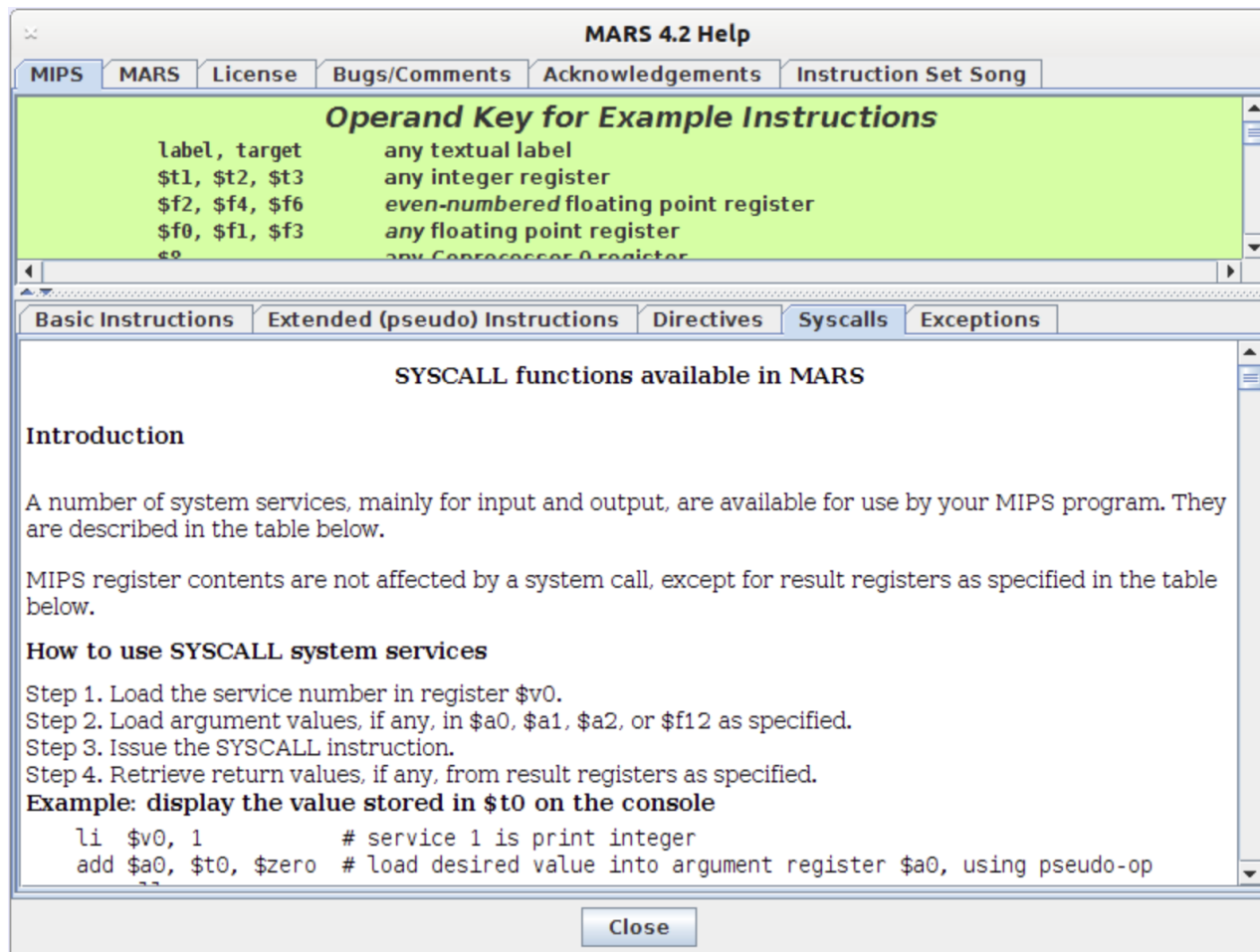
Help Menu



- Much of the information in the help window is available from popups that automatically appear while editing. The major exception is codes for the system calls.

System Calls

This help window screenshot shows the tab selections to bring up the system call information.



Edit/Execute Bar

- The edit/execute pane is a tabbed pane.
- Clicking on the "Edit" tab displays the MIPS assembly language editor.
- Clicking on the "Execute" tab displays assembly and run time information.

Registers Pane

- The registers pane is a tabbed pane. Unless you are writing systems or floating point programs you usually just leave the "Registers" tab selected. This displays the integer registers of the simulated MIPS processor.

Register Naming Conventions

- \$zero constant zero
- \$at reserved by assembler
- \$v0, \$v1 for parameter passing
- \$a0 to \$a3 for arguments
- \$t0 to \$t9 temporary registers (not saved by callee)
- \$s0 to \$s7 registers (saved by callee)
- \$gp global pointer
- \$sp stack pointer
- \$ra return address

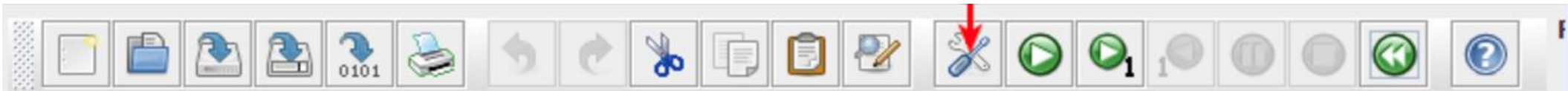
Mars Messages/Run I/O Pane

The Mars Messages/Run I/O pane is a tabbed pane.

- Clicking on the "Mars Messages" tab displays assembler and some run time messages.
- Clicking on the "Run I/O" tab displays program I/O and some run time messages.

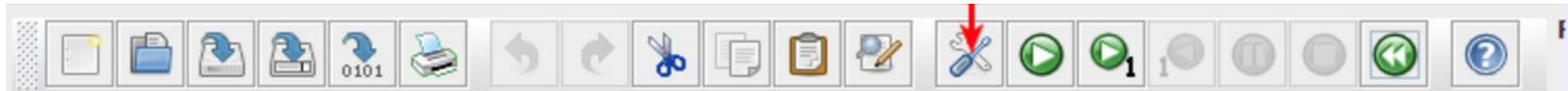
Assembling a File

- Programs can be assembled by clicking "Assemble" Button



Assembling a File

- Programs can be assembled by clicking "Assemble" Button



- After assembling the program, the Messages/Run I/O pane displays errors if there are any.
- If assembly is successful the Messages/Run I/O pane displays a message indicating that assembly was successful.

Assembling a File (Contd.)

- Execute" tab of the Edit/Execute pane is automatically selected.

The screenshot shows the MARS MIPS assembler interface with the 'Execute' tab selected. The 'Text Segment Contents' pane displays the following assembly code:

Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x24020004	addiu \$2,\$0,0x00000004	29: li \$v0, 4 # issue prompt
☐	0x00400004	0x3c011001	lui \$1,0x00001001	30: la \$a0, prompt
☐	0x00400008	0x34240000	ori \$4,\$1,0x00000000	
☐	0x0040000c	0x0000000c	syscall	31: syscall
☐	0x00400010	0x24020005	addiu \$2,\$0,0x00000005	33: li \$v0, 5 # get n from user
☐	0x00400014	0x0000000c	syscall	34: syscall
☐	0x00400018	0x00028021	addu \$16,\$0,\$2	35: move \$s0, \$v0
☐	0x0040001c	0x24110000	addiu \$17,\$0,0x000000...	37: li \$s1, 0 # sum = 0
☐	0x00400020	0x24120000	addiu \$18,\$0,0x000000...	38: li \$s2, 0 # i = 0
☐	0x00400024	0x0212082a	slt \$1,\$16,\$18	40: blt \$s0, \$s2, endf # exit loop if n < i
☐	0x00400028	0x14200003	bne \$1,\$0,0x00000003	

The 'Labels' pane shows the following labels and addresses:

Label	Address
(global)	
main	0x00400000
sums.s	
for	0x00400024
endf	0x00400038
prompt	0x10010000
result1	0x10010022
result2	0x10010038
newline	0x10010046

The 'Data Segment Contents' pane shows the following data segment:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x61656c50	0x65206573	0x7265746e	0x70206120	0x7469736f	0x20657669	0x65746e69	0x3a726567
0x10010020	0x68540020	0x75732065	0x666f206d	0x65687420	0x72696620	0x00207473	0x746e6920	0x72656765
0x10010040	0x73692073	0x000a0020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

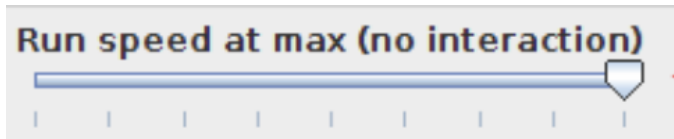
The 'Data Segment Contents' pane also includes a dropdown menu for the selected address (0x10010000 (.data)) and checkboxes for 'Hexadecimal Addresses', 'Hexadecimal Values', and 'ASCII'.

Running a Program

- Program is run by clicking on the "Go" go button.



- We can run the program at specified speed using this option – Variable Speed Execution



MIPS32 Assembly Code Layout

```
.text                #code section
.globl main          #starting point, must be global
main:
    #user program code goes here
.data                #data section
    #user program data goes here
```

Assembler Directives

- a) `.text` - specifies the user text segment, which contains the instruction.
- b) `.data` - specifies the data segment, where all the data items are defined.
- c) `.globl sym` – specifies that the symbol "sym" is global and can be referred from other files.
- d) `.word w1, w2,...,wn` – Stores the specified 32-bit number in successive memory words.
- e) `.half h1,h2,...,hn` – Stores the specified 16-bit numbers in successive memory half-words.

Assembler Directives

- f) `.byte b1, b2,..., bn` – Stores the specified 8-bit numbers in successive memory bytes.
- g) `.ascii str` – Stores the specified memory(in ASCII Code), but do not null-terminate it.
- h) `.ascii str` – Stores the specified string in memory(in ASCII code) and null-terminate it
- i) `.space n` – Reserves space for n successive bytes in memory.

Example Programs

References

- 1) <http://courses.missouristate.edu/kenvollmar/mars/papers.htm>
- 2) <https://www.youtube.com/playlist?list=PL5b07qlmA3P6zUdDf-o97ddfpvPFuNa5A>
- 3) <https://www.ece.ucdavis.edu/>
- 4) <https://www.d.umn.edu/~gshute/mips/Mars/Mars.html>